

第19讲

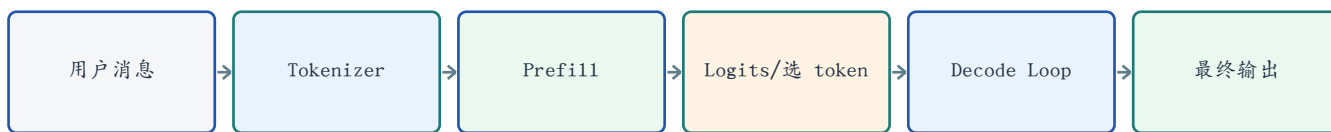
Inference：按下“发送”以后

大语言模型内部到底发生什么？

本讲主线

把一次 LLM

推理从用户输入一直追踪到最终输出：消息拼接、Tokenizer、Prefill、Transformer、KV
Cache、Logits、采样、Decode Loop 与停止条件。



一次典型 decoder-only LLM inference 的高层数据流

学习定位：理解模型服务与 Agent 系统的“运行时”，为后续 Temperature、KV
Cache、Latency、Context Engineering 打基础。

1. 本讲学完，你应该能回答什么？

Training 与 Inference 的根本区别是什么？

你输入一句话以后，为什么服务器并不是直接把“字符串”送进 Transformer？

Prefill 与 Decode 是什么？为什么它们的计算形态不同？

为什么模型每次只产生一个 next token，却最终能输出一整段话？

Logits、probability、sampling 分别是什么？

KV Cache 为什么可以避免重复计算过去的 Key / Value？

TTFT、TPOT / inter-token latency、throughput 分别在衡量什么？

为什么 Agent 一旦多轮调用 LLM，token、延迟和成本会被系统性放大？

一句话预告

Inference 就是：在参数基本固定的模型上，把当前上下文变成 next-token 概率分布，选出一个 token，把它接回上下文，再重复。

2. Training 和 Inference：一个“改参数”，一个“用参数”

维度	Training	Inference
目标	通过数据和 loss 学习参数	利用已学好的参数处理当前请求
反向传播	需要	通常不需要
参数更新	会	通常不会
输入	大量训练 batch	当前 prompt / conversation
输出	梯度、更新后的 checkpoint	logits、token、最终回答
主要成本	训练算力和显存巨大	每个请求都产生实时延迟与服务成本

如果把训练比作“多年学习形成大脑连接”，Inference 更像考试时真正使用已经形成的能力。考试过程中通常不会再做大规模反向传播重新训练自己。

不要混淆

RAG、工具调用、Agent Memory 可以在 inference 阶段改变模型“看到的上下文”，但这和直接修改模型参数不是同一件事。

3. 你按下“发送”时，真正进入模型的并不只有你刚写的一句话

聊天产品通常先构造一个完整请求。这个请求可能包含：

System message / developer instructions;

你的当前 User message;

之前若干轮对话历史;

工具定义、JSON schema 或 function description;

RAG 检索结果、上传文件片段、memory 等额外上下文。

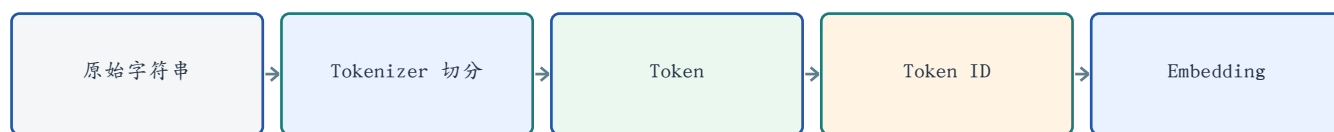
这些内容经过 Chat Template 后，会被串成模型能识别的 token 序列。模型看到的并不是 UI 里的气泡，而是一段有特殊标记的序列。

```
<system> ... </system> <user> ... </user> <assistant>
```

[Agent Systems 连接](#)

一个 Agent 的 system prompt、工具列表、memory、其他 Agent 消息，最后都会占用 context tokens。运行时系统设计会直接改变模型实际收到的输入。

4. 第一步：Tokenizer 把文字变成 Token IDs



例如“我喜欢研究AI。”不会以 Python 字符串形式直接进入神经网络，而会先被 tokenizer 转换成一串整数 ID。

[12873, 245, 9812, 503, ...]

这些整数本身没有语义几何结构；它们只是词表地址。随后通过 Embedding Matrix 查表，才变成 Transformer 可以计算的向量。

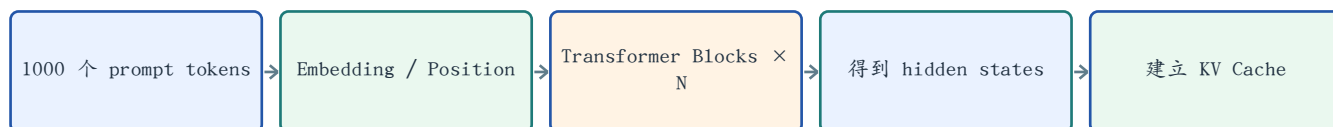
回忆第2-3讲

Token 决定“切成哪些离散单位”；Embedding 决定“这些单位以什么向量进入神经网络”。

5. 第二步：Prompt 进入 Prefill

Prefill 可以先理解成：模型一次性处理当前已经存在的整段上下文。

假设 prompt 有 1000 个 token，模型会让这些 token 经过 Embedding、位置编码以及一层层 Transformer Block，并计算每个位置所需要的隐藏表示和注意力状态。



为什么叫 prefill？因为它是在真正逐 token 生成之前，把“已有上下文”先填入模型的运行状态。

直觉

Prefill 像先把整份题目、资料和历史记录快速读一遍；Decode 则是从此以后一个字一个字地往答案里写。

6. Prefill 为什么可以高度并行？

虽然 Causal Mask 不允许位置 i 读取未来 token，但 prompt 中所有 token 都已经已知，因此训练/Prefill 时可以把很多位置放在一个大矩阵中一起计算。

$$Q = X W_Q, K = X W_K, V = X W_V$$

这里 X 可以同时包含几百、几千甚至更多 token 的表示。GPU 擅长这种大规模矩阵乘法，因此 Prefill 通常能够充分利用并行计算。

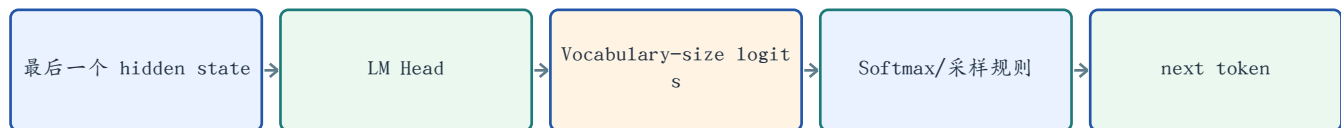
但是注意：Self-Attention 的计算/内存开销会随上下文长度快速增长，标准 attention 中与 token-pair 相关的部分具有约 $O(n^2)$ 的规模特征。

重要区分

“因果”不等于“必须串行计算 prompt”。Causal Mask 只规定谁能看谁；只要整段 prompt 已知，很多位置仍能并行算。

7. Prefill 的最后：从最后位置得到 next-token Logits

Decoder-only LLM 的目标是根据已有前缀预测下一个 token。Prefill 完成后，最关键的是当前最后一个位置的 hidden state。



如果词表大小是 100,000，那么 LM Head 会给出约 100,000 个候选 token 的分数，这组原始分数就叫 logits。

$$\text{logits} = \mathbf{h_last} \mathbf{W_vocab} + \mathbf{b}$$

logits 还不是概率。后续通常通过 Softmax 以及 Temperature / Top-k / Top-p 等规则形成实际采样分布。这些细节会在下一讲专门展开。

8. Logits：模型不是直接“想出一个词”，而是形成一个候选分布

假设某一步候选 token 的简化分数是：

候选 token	logit	Softmax 后概率（示意）
“模型”	4.2	0.55
“系统”	3.7	0.33
“天气”	1.2	0.03
其他	...	0.09

然后 runtime 根据解码策略选择一个 token。最简单的是 Greedy Decoding：直接选择概率最大的那个；也可以进行随机采样。

科研直觉

同一个模型参数不变，只改变 sampling policy，也可能显著改变输出多样性、稳定性和错误相关性。以后做 Agent 实验时，这是必须控制的变量。

9. 第一个新 Token 产生以后：进入 Decode Loop

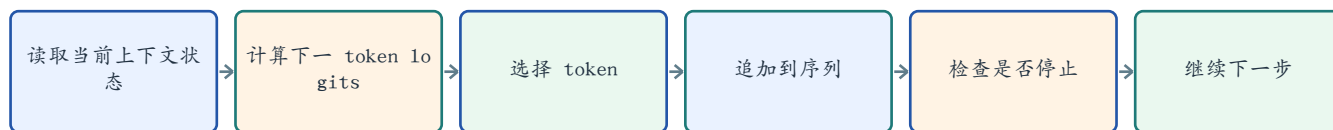
假设模型从 prompt 后预测出了 token A。接下来真正关键的一步是：

Prompt + A -> token B

然后：

Prompt + A + B -> token C

于是形成经典的 Autoregressive Decode Loop：



因此你看到回答在屏幕上一个 token 一个 token 流出来，不是 UI 特效，而与自回归生成机制直接相关。

10. 为什么 Decode 比 Prefill 更“串行”？

生成第 $t+1$ 个 token 之前，系统必须先知道第 t 个 token 到底是什么。因为第 t 个 token 会进入下一步上下文。

token_1 -> token_2 -> token_3 -> ...

所以单条序列的 decode 存在天然的时间依赖：后一步依赖前一步的实际采样结果。

阶段	已知 token	并行特征	直觉
Prefill	整段 prompt 都已知	可对很多位置并行	把已有材料一起读入
Decode	未来 token 尚未产生	单条序列按 token 依次推进	写一个词，才能知道下一步前缀
系统含义	这也是为什么“生成很长的回答”通常会持续占用服务时间：输出 token 数直接增加 decode steps。		

11. 一个巨大的浪费问题：难道每生成一个 token，都要把过去全部重新算一遍？

如果完全朴素地实现，生成第 1001 个 token 时会重新处理前 1000 个 token；生成第 1002 个时又把前 1001 个重新处理。大量过去的 Key / Value 都会被重复计算。

但对于固定的过去 token，它们在某一层产生的 K 和 V 不会因为“未来多生成一个 token”而改变。于是可以把这些 K/V 保存下来。

这就是：

$$\text{KV Cache} = \text{Key/Value Cache}$$

下一步只需要为新 token 计算新的 Q/K/V，然后把新 K/V 追加到缓存中，再让新 Query 去读取整个历史 K/V。

12. KV Cache：把过去的 Attention 状态“存档”



对每一层、每个 relevant attention head，都需要保存历史 K/V。因此 KV Cache 会随：

上下文长度增加；

层数增加；

K/V head 数增加；

KV 向量维度增加；

同时服务的序列数增加。

这也是为什么长上下文服务不仅是“计算问题”，还是非常现实的显存/内存管理问题。

先建立直觉

KV Cache 不是“模型长期记忆”，它只是当前 inference session 的计算缓存。会话结束后可以被释放；它和 Agent Memory 是不同概念。

13. 为什么 KV Cache 能节省计算？

假设 prompt 长度为 3，随后生成 4 个 token。朴素方法在每一步 decode 都重新计算所有历史 token 的 K/V；缓存方法只计算新 token 的 K/V。

时刻	朴素 K/V 投影行数	使用 Cache 新增行数
Prefill 长度 3	3	3
生成第1个 token 后	4	1
生成第2个 token 后	5	1
生成第3个 token 后	6	1
生成第4个 token 后	7	1

这只是教学上的简化计数。真实 Transformer 还有 Attention 读取、FFN、norm、memory movement 等大量开销。但它准确体现了一个关键思想：

不要重复计算过去不会变化的 K/V。

14. 配套实验：Prefill + Decode + KV Cache

材料包中的 Python 代码实现了一个极小的单头 causal attention 玩具模型。它会：

- 对 prompt 做一次 Prefill;
- 保存 K_cache / V_cache;
- 通过 LM Head 产生下一 token logits;
- 使用 Greedy 选择 token;
- 每一步只为新 token 计算新的 Q/K/V;
- 统计“朴素重算”和“使用 KV cache”的 K/V 投影行数。

固定随机种子的实际输出摘录：

```
PROMPT: [' ', 'I', 'like']
PREFILL sequence length: 3
KV cache rows after prefill: 3
last hidden shape: (8,)

DECODE step 1
next token: I
top probability: 0.21883535556417555
context length now: 4

DECODE step 2
next token: I
top probability: 0.21864397751274692
context length now: 5

DECODE step 3
next token: I
top probability: 0.21098332222805655
context length now: 6
```

注意

这是教学模型，不是 GPT 的真实实现。目的只是让 Prefill / Decode / KV Cache 变成看得见的数组和循环。

15. 什么时候停止生成?

Decode Loop 不会无限运行。常见停止条件包括:

EOS token: 模型生成 End-of-Sequence 特殊 token;

Stop sequence: runtime 检测到指定字符串或 token 模式;

max_new_tokens: 达到最大输出 token 数;

工具调用结构完成: 例如 function-call JSON 已闭合, runtime 转去执行工具;

外部中断: 用户取消、超时、服务策略终止。

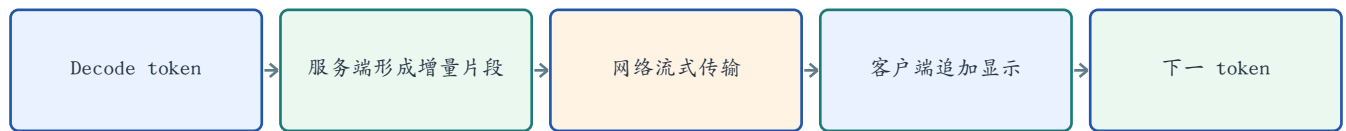
因此“停止”既可能来自模型生成的特殊 token, 也可能来自模型外部的 runtime 规则。

Agent 连接

Agent Tool Calling 常常不是“模型直接执行工具”, 而是模型先生成结构化意图, runtime 识别后暂停文本生成, 执行工具, 再把结果作为新上下文送回模型。

16. Streaming：为什么你能边生成边看到文字？

模型每完成一个或一小批 token，服务端就可以立即把新增内容通过流式协议发送给客户端。客户端把它追加显示，于是你看到“打字机式”输出。



Streaming 并不会让模型本身变聪明，但它能显著改善用户感知延迟：即使完整回答还要几秒钟，用户可以更早看到第一个 token。

这就引出了两个非常重要的延迟指标。

17. TTFT、TPOT、Throughput：读系统论文必须认识的指标

指标	全称 / 含义	直觉
TTFT	Time To First Token	从请求发出到第一个输出 token 出现要多久
TPOT / ITL	Time Per Output Token / Inter-Token Latency	首 token 之后，相邻输出 token 间隔多长
End-to-end latency	从请求到完整回答结束	用户等待完整结果多久
Throughput	单位时间处理的 token / request 数	整个服务系统吞吐能力

粗略理解：长 prompt 往往显著影响 Prefill 和 TTFT；长输出则会增加 Decode steps 和完整响应时间。实际系统还会受到排队、batching、并行策略和硬件等因素影响。

18. 为什么 Prefill 和 Decode 的硬件瓶颈可能不同？

Prefill 通常包含较大的矩阵计算，可以形成高并行度的矩阵-矩阵乘法；Decode 时每一步只增加少量新 token，却要反复读取大量模型权重和 KV cache。

因此在很多实际部署中：

Prefill 往往更容易把 GPU 的大规模计算单元“喂满”；

Decode 常常更容易受到 memory bandwidth、kernel launch、batch size 和 latency 约束；

但具体瓶颈依赖模型结构、序列长度、batch、量化、硬件和 serving stack，不能机械套用。

科研表达

更严谨的说法是“prefill 与 decode 具有不同的计算形态和性能瓶颈”，而不是简单说某一阶段永远 compute-bound / memory-bound。

19. Batching：为什么服务器会把多个用户请求一起算？

GPU 喜欢“大块矩阵”。如果每次只服务一个很短请求，硬件利用率可能很低。Serving 系统会尝试把多个请求组织成 batch，一起执行部分计算。

方式	特点	问题
Static batching	先凑一批，再一起跑	不同请求长度不一，容易有人等别人
Continuous batching	请求在 decode 过程中动态加入/退出 batch	调度更复杂，但在线服务效率更高

因此单个用户体验和整个服务吞吐之间存在系统级权衡：batch 大一些通常更利于吞吐，但可能增加排队或单请求延迟。

AI Systems Science	“同一个模型为什么在不同服务框架里延迟差很多？”这不是模型能力问题，而是系统调度、缓存、并行、批处理和硬件利用问题。
--------------------	--

20. Prompt 越长，Inference 成本发生什么变化？

更长上下文至少会带来三类代价：

Prefill 计算更多：需要处理更多输入 token；

KV Cache 更大：需要保留更多历史 K/V；

Attention 读取范围更长：每个新 token 的 Query 需要与更长历史状态交互。

所以“把所有历史都塞进 prompt”不是免费的。Agent 系统如果每轮都把完整工具日志、长 memory 和所有 Agent 消息重新注入，上下文成本会迅速增长。

Context Engineering

的动机

后续你会看到摘要、检索、memory selection、context pruning

等方法，本质上都在解决“有限上下文与有限推理预算下，应该让模型看到什么”。

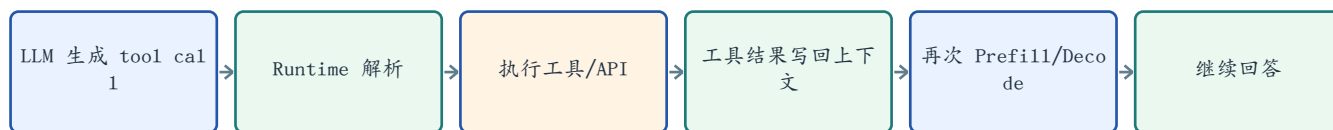
21. API 账单为什么经常分 Input Tokens 和 Output Tokens?

从运行时看，输入 token 主要进入 Prefill；输出 token 则一个个经历 Decode。两者都消耗资源，但计算形态不同，因此服务商经常分别计价。

Token 类型	主要阶段	对系统的典型影响
Input / prompt tokens	Prefill	影响上下文处理、KV cache 初始大小、TTFT
Output / completion tokens	Decode	增加串行生成 step、响应时长、持续占用 cache

这也解释了为什么 Agent 的“反复思考 + 多轮工具调用 + 多 Agent 讨论”会很快增加 API 成本：每一轮都可能重新引入大量 input tokens，并产生新的 output tokens。

22. Tool Calling 时，Inference Loop 会变成什么？



例如模型生成:

```
{"tool": "calculator", "arguments": {"expression": "37*91"}}
```

runtime 不是把这段 JSON 当最终答案，而是执行计算器，得到结果后形成新的 tool message，再交给模型继续推理。

于是 Agent 推理不再只是单个 decode loop，而是多个 LLM inference episode + 外部动作之间的循环。

失败模式

工具 schema 过长、工具结果太冗长、循环次数过多、每轮重新塞入完整历史，都会直接放大 token、latency 与 cost。

23. Multi-Agent：一次“讨论”背后可能是多少次 Inference？

假设 4 个 Agent 先独立回答，再互相看对方答案，最后由 Judge 聚合。表面上只是一次“群聊”，底层可能是：

阶段	LLM 调用次数示例	额外上下文
4 个 Agent 独立回答	4	同一个任务 prompt
互评一轮	4	任务 + 其他 Agent 输出
二次修订	4	更长的群体历史
Judge 聚合	1	几乎全部候选与讨论

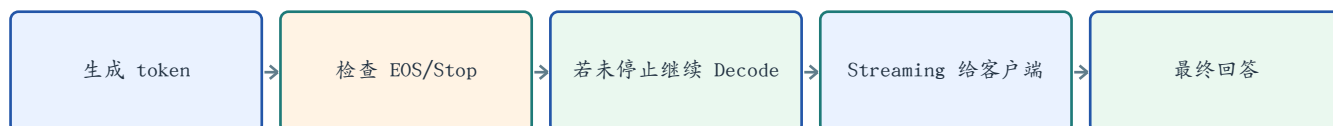
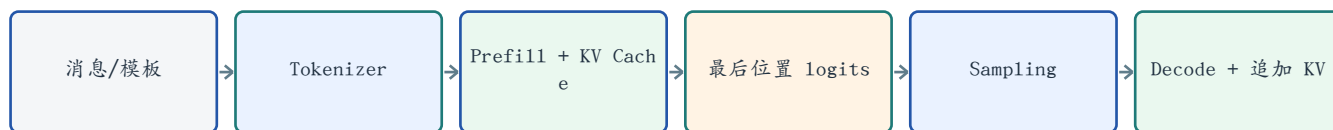
合计可能已经 13 次 LLM inference，而且后期调用的 prompt 会比最初更长。

科研含义	Multi-Agent 性能不能只报告 accuracy。至少还要报告 model calls、input/output tokens、latency、API cost，并做 compute-matched baseline。
------	---

24. 常见误区

误区	为什么不对
“模型一次就把整段答案算出来，只是慢慢显示”	自回归 decoder 通常逐 token 生成；后 token 依赖前 token 的实际结果。
“KV Cache 就是模型记住了这次对话”	它是运行时计算缓存，不等于长期记忆系统。
“Prefill 也是一个 token 一个 token 串行跑”	已知 prompt 的很多位置可以并行计算，只受 causal 可见性约束。
“Logits 就是概率”	logits 是未归一化分数；要经过 Softmax / sampling policy。
“Streaming 能让模型算得更快”	它主要让结果更早显示，不必然减少底层计算。
“长上下文只多花一点 token 费”	还可能扩大 prefill、attention、KV cache、latency 等系统开销。

25. 把一次 LLM Inference 完整串起来



工具调用时，这个流程还会被外部 runtime 打断并再次进入新的 inference episode

到这里你应该能看到：所谓“一次 ChatGPT

回答”，底层不是一个黑箱动作，而是一串结构清晰的运行时阶段。

User text -> formatted context -> token IDs -> embeddings -> prefill -> logits -> select next token -> decode loop -> stop

26. 与后续课程怎么连接？

后续主题	本讲已经打下的基础
Temperature / Top-k / Top-p	本讲知道了 logits 之后还需要 “选择 token”
Context Window	本讲知道 input tokens 会进入 prefill 和 KV cache
KV Cache / Serving Systems	本讲已经建立 cache、TTFT、TPOT、batching 的第一层直觉
Quantization / MoE	会进一步改变每次 forward 的内存、计算和吞吐
RAG / Tool Use / Agent	会把一次 inference 扩展成多轮 runtime loop

本讲最低闭环	你不需要会写 CUDA kernel，但必须能解释：为什么 prompt 很长会慢、为什么输出很长也会慢、为什么 KV cache 有价值、为什么 Agent 的多轮调用会放大成本。
--------	---

27. 小实验：建议你亲手做一次

打开材料包中的 `04_inference_kv_cache_demo.py`，先直接运行，再尝试做三处修改：

把 `prompt_ids` 从 3 个 token 改成 5 个，观察 Prefill cache 行数；

把 `max_new_tokens` 从 4 改成 10，观察 naive/cache 计数比如何变化；

把 Greedy 选择改成 `np.random.choice` 按概率采样，观察每次生成是否可能不同。

你不需要理解代码里的所有矩阵细节。真正要观察的是：

Prefill 一次处理已有上下文；Decode 每次只推进一个新 token；KV Cache 让过去的 K/V 不再被重复投影。

28. 本讲小结

Inference 是在参数基本固定时处理当前请求，不做大规模反向传播。

聊天消息会先被模板化、tokenize，再进入 Transformer。

Prefill 负责处理已有上下文；Decode 负责逐 token 生成。

最后 hidden state 经 LM Head 变成全词表 logits。

选择一个 token 后，把它加入上下文，再重复 next-token prediction。

KV Cache 保存历史 K/V，避免每一步都重新计算过去的 K/V 投影。

TTFT 关注首 token，TPOT / ITL 关注持续生成速度，throughput 关注整个系统吞吐。

Agent / Multi-Agent 会把单次 inference 扩展成多轮模型调用与环境交互，成本必须系统记账。

核心句：

Prefill 是“读完已有上下文”；Decode 是“一个 token 一个 token 往前写”；KV Cache 是“过去算过的 K/V 别再重复算”。

29. 练习题

- 1) 为什么 prompt 已知时, Causal Attention 仍可以在 Prefill 中并行处理多个位置?
- 2) Logits 和 probability 有什么区别?
- 3) 为什么 decoder-only LLM 的单条输出序列通常必须逐 token 生成?
- 4) KV Cache 保存的是什么? 它为什么不是 Agent 的长期记忆?
- 5) 一个请求 TTFT 很高, 可能和哪些阶段有关?
- 6) 为什么长 output 会显著增加 end-to-end latency?
- 7) 为什么 Multi-Agent 实验必须报告 input/output token 与 model calls?
- 8) 如果一个 Agent 每轮都把完整工具日志重新塞回 prompt, 系统层面可能出现什么问题?

答案见独立《练习题与答案》PDF。